

TPB2 : Création d'application avec Flutter

Table des matières

1) Problématique.....	1
2) Prérequis	1
3) Exercices (Flutter)	3
4) Exercices (DART).....	5
5) Packages Flutter	34
5) Flutter Dark Mode	38

1) Problématique

L'objectif de ce TP est de concevoir et développer une application à l'aide de Flutter, en utilisant Android Studio et Visual Studio Code comme environnements de développement. Le langage Dart sera utilisé pour programmer l'application.

2) Prérequis

Afin de mener à bien ce TP nous allons avoir besoin de 4 outils :

- **Visual Studio Community 2026** :

<https://visualstudio.microsoft.com/fr/vs/community/>

- **Visual Studio Code** : <https://code.visualstudio.com/Download>

- **SDK Flutter** : <https://docs.flutter.dev/install/manual>

- **Android Studio** : <https://developer.android.com/studio>

Il va falloir installer chaque outil et configurer certains. Par exemple, il faudra ajouter le kit de développement C++ pour Visual Studio Community et il faudra installer l'outil CMD Android Tools.

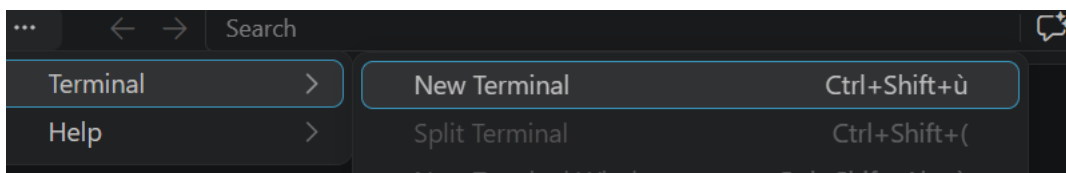
On peut vérifier grâce à la commande flutter doctor si tout est bien configuré :

```
Microsoft Windows [version 10.0.26200.9278]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Ewan Alexandre>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.2, on Microsoft Windows [version 10.0.26200.9278], locale fr)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.9.2)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!
```

Dans VSCode, on commence d'abord par ouvrir le terminal comme ceci :



Ensuite, dans le terminal on se place sur le dossier du projet avec **cd** :

```
● PS C:\Users\Ewan Alexandre> cd develop
○ PS C:\Users\Ewan Alexandre\develop> |
```

Ensuite on utilise **flutter create nom_application** pour créer la base de notre application :

```
● PS C:\Users\Ewan Alexandre\develop> flutter create mon_app
Creating project mon_app...
Resolving dependencies in `mon_app`...
Downloading packages...
Got dependencies in `mon_app`.
Wrote 131 files.

All done!
```

Puis cette fois-ci on se place dans le dossier de l'application avec **cd** puis on utilise **code .** pour ouvrir le projet dans VSCode :

```
PS C:\Users\Ewan Alexandre\develop> cd mon_app
PS C:\Users\Ewan Alexandre\develop\mon_app> code .
```

Une page devrait alors s'ouvrir, avec le projet créé ouvert. On va chercher le main.dart dans le dossier lib de cette manière :



The image shows a file explorer in VS Code with the 'lib' folder expanded. The file 'main.dart' is highlighted in the list. To the right of the file list, there are line numbers 6, 7, and 8.

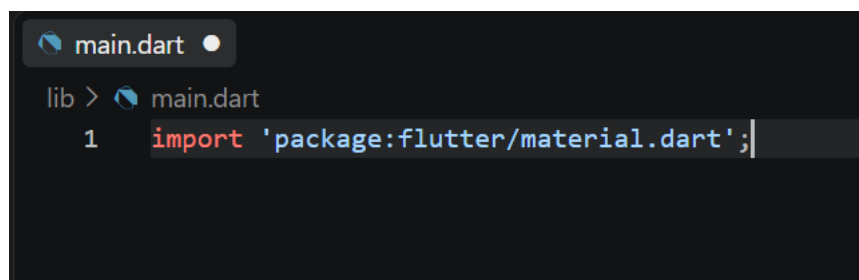
Nous allons désormais supprimer son contenu par défaut pour pouvoir commencer à créer notre application de zéro.

3) Exercices (Flutter)

1. Page vide

Nous pouvons à présent commencer à travailler sur notre projet, notre première étape sera de créer une page vide.

Pour créer une page vide on s'aide du guide qui nous est donné, celui-ci nous dit que l'on doit d'abord importer le package Material de Flutter comme ceci :



The image shows a code editor window with a tab labeled 'main.dart'. The editor content shows the path 'lib > main.dart' and the first line of code: `1 import 'package:flutter/material.dart';`

Par la suite on nous demande d'ajouter le **void main()** qui permettra d'exécuter le code, et à l'intérieur de celui-ci on ajoute la fonction **runApp()** de cette manière :

```
import 'package:flutter/material.dart';

void main() {

  runApp()

}
```

Parfait ! Nous avons maintenant une base qui permettra d'effectuer la suite du TP.

2. Créer un fond de couleur

Pour pouvoir créer un fond de couleur pour notre application nous allons utiliser le widget **Container()** qui représente une boîte simple. Nous allons attribuer la couleur rouge au container avec le paramètre « color : » puis on sélectionne l'attribut « red » de la classe « Colors » avec Colors.red et on attribue la classe au paramètre colors : comme ceci :

```
Container(
  color: ■ Colors.red,
),
```

Les , sont l'équivalent des ; elles permettent d'indiquer la fin du ligne au sein d'un argument donc color est attribué à container et la , après container indique que celui-ci est fini. Le code entier ressemble à ceci :

```
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(
    Container(
      color: ■ Colors.red,
    ),
  );
}
```

Et le résultat est le suivant :



3. Créer du texte centré

Pour créer un texte centré nous allons utiliser le paramètre **child** pour ajouter un enfant au Container, cet enfant sera un widget **Center**, de type **const**, qui lui-même aura pour enfant un **Text** avec un paramètre de direction qui précise que le texte va de gauche à droite (« ltr ») :

```
Run | Debug | Profile
void main() {
  runApp(
    Directionality(
      textDirection: TextDirection.ltr,
      child: Container(
        color: Colors.red,
        child: const Center(
          child: Text(
            'Hello, World!',
          ), // Text
        ), // Center
      ), // Container
    ), // Directionality
  );
}
```

4) Exercices (DART)

Pour effectuer les exercices DART, nous allons utiliser VCC pour créer notre code et le tester et les documentations DART pour nous aider à répondre aux questions.

Exercice 1 :

On commence par importer l'outil permettant de capter les entrées claviers : « dart :io ». On demande à l'utilisateur son nom puis celui-ci pourra l'indiquer et la réponse sera contenu dans la variable nom. Pareil pour ageTexte, cependant ageTexte est considéré comme un texte on utilise donc un int.parse pour convertir le texte en entier puis on attribue le résultat à une nouvelle variable age. Puis on déclare une variable anneesRestantes qui calcule combien d'années il reste à l'utilisateur avant qu'il ait 100 ans. A la fin on affiche un message qui lui indique donc le résultat et on utilise \$ pour afficher le contenu des variables.

```
import 'dart:io';

Run | Debug | Profile
void main() {
  print("Entrez votre nom :");
  String nom = stdin.readLineSync();

  print("Entrez votre âge :");
  String ageTexte = stdin.readLineSync();

  int age = int.parse(ageTexte);
  int anneesRestantes = 100 - age;

  print("Bonjour $nom, il te reste $anneesRestantes ans avant d'avoir 100 ans !");
}
```

Ensuite on lance le code avec **dart run lib/main.dart**.

Résultat :

```
Entrez votre nom :
Ewan
Entrez votre âge :
19
Bonjour Ewan, il te reste 81 ans avant d'avoir 100 ans !
```

Exercice 2 :

Ici on demande à l'utilisateur d'entrer un nombre, on récupère ce nombre et on le stocke dans la variable saisie, ensuite on convertit ce « faux » nombre en vrai nombre avec le int.parse. Par la suite on effectue un test pour vérifier si la division du nombre choisit par 2 laisse un reste de 0 ou non, si oui alors on indique que c'est pair, sinon on indique que c'est impair.

```
import 'dart:io';

Run | Debug | Profile
void main() {
  print("Entrez un nombre :");
  String saisie = stdin.readLineSync!;

  int nombre = int.parse(saisie);

  if (nombre % 2 == 0) {
    print("Le nombre $nombre est pair.");
  } else {
    print("Le nombre $nombre est impair.");
  }
}
```

Résultats :

```
Entrez un nombre :
20
Le nombre 20 est pair.
```

```
Entrez un nombre :
19
Le nombre 19 est impair.
```

Exercice 3 :

On crée une liste a, on fait une boucle qui vérifie si le nombre de chaque indice est strictement inférieur à 5, si oui alors le nombre est affiché, sinon rien n'est affiché. Cela affichera donc que les éléments inférieurs à 5.

```
void main() {
  List<int> a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];

  for (int nombre in a) {
    if (nombre < 5) {
      print(nombre);
    }
  }
}
```

Résultat :

```
PS
1
1
2
3
```

Exercice 4 :

Ici on demande un nombre à l'utilisateur, on convertit sa réponse en entier, on fait une boucle qui part de 1 (i est le compteur) et qui ne s'arrête pas tant que la condition i inférieur ou égal au nombre choisi par l'utilisateur est vrai puis i augmente de 1 à chaque réitération avec l'incrémementation i++.

Ensuite on vérifie que le nombre choisi par l'utilisateur divisé par i donne un reste de 0. Si c'est le cas alors i est un diviseur du nombre choisi et donc on l'affiche, sinon on l'affiche pas.

```
import 'dart:io';

Run | Debug | Profile
void main() {
  print("Entrez un nombre :");
  String saisie = stdin.readLineSync();

  int nombre = int.parse(saisie);

  for (int i = 1; i <= nombre; i++) {
    if (nombre % i == 0) {
      print(i);
    }
  }
}
```

Résultat :

```
Entrez un nombre :
10
1
2
5
10
```

Exercice 5 :

On déclare les deux listes, on crée une liste qui va récupérer les éléments communs des deux listes. On effectue une boucle dans une boucle ce qui va permettre de vérifier chaque indice un par un de chaque liste en même temps. Le `for (int nombreA in a)` est une boucle for each donc ça parcourt

tous les éléments de la liste. Pareil pour le B du coup. Dans la boucle du B on vérifie donc que le nombre présent à l'indice de A est le même que celui présent au même endroit à l'indice de B. Si c'est le cas alors le nombre est ajouté à la liste « communs », si ça ne l'est pas alors rien ne se passe. Puis lorsque tous les indices ont été parcourus les deux boucles s'arrêtent et il ne reste plus qu'à afficher la liste des nombres communs. Une vérification se fait pour voir si un nombre en commun entre les deux listes n'est pas déjà dans la liste commune. Par exemple, imaginons qu'à l'indice 4 des deux boucles le nombre est 5, et que c'est aussi le cas pour l'indice 5, alors dans ce cas cela ne vas pas ajouter deux fois le nombre 5 dans la liste des communs.

```
void main() {  
    List<int> a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];  
    List<int> b = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];  
  
    List<int> communs = [];  
  
    for (int nombreA in a) {  
        for (int nombreB in b) {  
            if (nombreA == nombreB) {  
                if (!communs.contains(nombreA)) {  
                    communs.add(nombreA);  
                }  
            }  
        }  
    }  
  
    print(communs);  
}
```

Exercice 6 :

Ici, la logique est plus difficile. En gros on demande un mot à l'utilisateur. On déclare une variable chaîneInverse vide qui va permettre de stocker lettre par lettre le mot inversé. Ensuite, on déclare le compteur qui prendre la valeur de la longueur de la chaîne (par exemple kayak = 5 donc i = 5) et enlève -1 car en fait ce sont les indices qui vont être parcourus donc si on commençait à 5 le code ne serait pas quoi faire car kayak va de 0 à 4 en termes d'indices. On décide donc d'enlever 1 au tout pour que ça comme bien avec la dernière lettre puis on décrémente jusqu'à que la condition $i \geq 0$ soit fausse. On fait supérieur ou égal à 0 pour aussi prendre l'indice 0 du nombre (donc la première lettre). Enfin, à chaque boucle la lettre présente à l'indice i est ajouté à la variable chaîneInverse, et une

vérification if se fait pour vérifier que la chaîne de base est bien égal à la chaîne inverse, si c'est le cas alors ça indique la chaîne est un palindrome sinon elle indique que ça n'en est pas un.

```
Run | Debug | Profile
✓ void main() {
    print("Entrez une chaîne :");
    String chaîne = stdin.readLineSync();

    String chaîneInverse = "";
    for (int i = chaîne.length - 1; i >= 0; i--) {
        chaîneInverse = chaîneInverse + chaîne[i];
    }

    if (chaîne == chaîneInverse) {
        print("La chaîne est un palindrome.");
    } else {
        print("La chaîne n'est pas un palindrome.");
    }
}
```

Exercice 7 :

On commence par déclarer la liste donné, on déclare une nouvelle liste « pairs », on utilise une boucle qui parcourt la liste donné et on fait une vérification pour savoir si le nombre est pair, si c'est le cas on rajoute le nombre à la liste pairs, sinon on ne fait rien. A la fin on affiche la liste.

```
void main() {
    List<int> a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100];
    List<int> pairs = [];

    for (int nombre in a) {
        if (nombre % 2 == 0) {
            pairs.add(nombre);
        }
    }

    print(pairs);
}
```

Résultats :

```
[4, 16, 36, 64, 100]
```

Exercice 8 :

Cette fois-ci, nous allons ajouter une bibliothèque en plus : `dart:math`

Cette bibliothèque permettra d'avoir accès à la fonction `random` qui nous sera utile pour ce jeu de pierre papier ciseau.

On démarre avec une liste de choix possibles : pierre / papier / ciseau.

Ensuite on demande à l'utilisateur de choisir parmi les trois (je fais au plus simple donc je ne fais aucune vérification) mais je force quand même les lettres en minuscules pour pas qu'il y ait des problèmes de match. Ensuite on attribue le choix à la variable `joueur`.

Ensuite on déclare un objet `random` qui servira d'adversaire.

On indique que la variable `ordi` va récupérer la valeur du choix de `random`, `random.nextInt` va en fait faire choisir un nombre entre 0 et 2 car le 3 indique en fait une limite à ne pas égaler ni franchir. Donc le robot va prendre un indice aléatoire, et par conséquent avoir un choix aléatoire.

On affiche par la suite au joueur ce qu'à choisi l'ordi puis on fait les vérifications :

Si le joueur et l'ordi ont eu le même choix : on affiche égalité

Sinon si le joueur a fait pierre et l'ordi ciseau ou joueur a fait papier et l'ordi a fait pierre ou le joueur a fait ciseau et l'ordi papier (On utilise `||` comme OU) alors le joueur gagne et dans n'importe quel autre cas il perd.

```

import 'dart:io';
import 'dart:math';

Run | Debug | Profile
void main() {
  List<String> choixPossibles = ['pierre', 'papier', 'ciseaux'];

  print("Votre choix (pierre, papier ou ciseaux) :");
  String joueur = stdin.readLineSync()!.toLowerCase();

  Random random = Random();
  String ordi = choixPossibles[random.nextInt(3)];

  print("L'ordinateur a choisi : $ordi");

  if (joueur == ordi) {
    print("Égalité !");
  } else if (
    (joueur == 'pierre' && ordi == 'ciseaux') ||
    (joueur == 'papier' && ordi == 'pierre') ||
    (joueur == 'ciseaux' && ordi == 'papier')
  ) {
    print("Vous avez gagné !");
  } else {
    print("L'ordinateur a gagné !");
  }
}

```

Résultat :

```

Votre choix (pierre, papier ou ciseaux) :
papier
L'ordinateur a choisi : pierre
Vous avez gagné !

```

Exercice 9 :

On importe la bibliothèque dart:math en plus de dart:io afin de pouvoir générer un nombre aléatoire avec l'objet Random. On génère le nombre

mystère entre 1 et 100 grâce à `random.nextInt(100) + 1` (car `nextInt(100)` donne un nombre entre 0 et 99, on décale donc de 1). On crée une variable compteur qui commence à 0 pour enregistrer le nombre de tentatives. Ensuite, on lance une boucle infinie `while (true)` qui ne s'arrête que lorsque le bon nombre est découvert. À chaque itération, l'utilisateur tape un nombre, on le convertit en entier avec `int.parse` et on ajoute 1 au compteur avec `compteur++`. Si sa valeur est plus petite que le nombre mystère, on lui indique que c'est trop bas, si elle est plus grande on indique que c'est trop haut. Enfin, si le nombre est le bon, on affiche un message de réussite avec le nombre total de suppositions effectuées et on quitte la boucle avec l'instruction `break`.

```
import 'dart:io';
import 'dart:math';

Run | Debug | Profile
✓ void main() {
    Random random = Random();
    int nombreMystere = random.nextInt(100) + 1;
    int compteur = 0;

    print("Bienvenue dans le jeu ! Devinez le nombre entre 1 et 100.");

    while (true) {
        stdout.write("Entrez votre supposition : ");
        String? saisie = stdin.readLineSync();
        if (saisie == null || saisie.isEmpty) break;
        int supposition = int.parse(saisie);
        compteur++;

        if (supposition < nombreMystere) {
            print("C'est trop bas !");
        } else if (supposition > nombreMystere) {
            print("C'est trop haut !");
        } else {
            print("Bravo, vous avez trouvé le nombre !");
            print("Nombre de suppositions : $compteur");
            break;
        }
    }
}
```

Résultat :

```
Bienvenue dans le jeu ! Devinez le nombre entre 1 et 100.  
Entrez votre supposition : 50  
C'est trop bas !  
Entrez votre supposition : 75  
C'est trop haut !  
Entrez votre supposition : 62  
C'est trop bas !  
Entrez votre supposition : 68  
Bravo, vous avez trouvé le nombre !  
Nombre de suppositions : 4
```

Exercice 10 :

Ici on nous demande d'écrire une fonction qui vérifie si un nombre est premier. Un nombre premier est un nombre divisible uniquement par 1 et par lui-même. On crée donc une fonction `estPremier(int nombre)` qui va renvoyer un booléen (true ou false). Si le nombre est inférieur ou égal à 1, il n'est pas premier donc on renvoie false. Ensuite on fait une boucle for qui démarre à 2 (car tous les nombres sont divisibles par 1) et qui va jusqu'au nombre choisi. Si à un moment le reste de la division `nombre % i == 0`, cela veut dire qu'on a trouvé un diviseur donc ce n'est pas un nombre premier, la fonction renvoie false. Si la boucle se termine sans rien trouver, c'est que le nombre est bien premier et on renvoie true. Dans le `main()`, on demande un nombre à l'utilisateur, on appelle la fonction et on affiche la phrase correspondante.

```
import 'dart:io';  
  
bool estPremier(int nombre) {  
  if (nombre <= 1) {  
    return false;  
  }  
  for (int i = 2; i < nombre; i++) {  
    if (nombre % i == 0) {  
      return false;  
    }  
  }  
  return true;  
}  
  
Run | Debug | Profile  
✓ void main() {  
  stdout.write("Entrez un nombre : ");  
  String saisie = stdin.readLineSync();  
  int nombre = int.parse(saisie);  
  
  if (estPremier(nombre)) {  
    print("$nombre est un nombre premier.");  
  } else {  
    print("$nombre n'est pas un nombre premier.");  
  }  
}
```

Résultats :

```
Entrez un nombre : 7
7 est un nombre premier.
```

```
Entrez un nombre : 12
12 n'est pas un nombre premier.
```

Exercice 11 :

Pour cet exercice, on crée une fonction qui reçoit une liste d'entiers et qui doit renvoyer une nouvelle liste contenant uniquement le premier et le dernier élément de cette liste. Dans la fonction `premierEtDernier`, on déclare une liste vide `resultat`. Pour obtenir facilement le premier élément on utilise la propriété `.first`, et pour le dernier on utilise `.last`. On les ajoute tour à tour à notre liste avec la méthode `add()`, puis on renvoie la liste finale. Dans le `main()`, on déclare la liste donnée dans l'exemple `a = [5, 10, 15, 20, 25]`, on passe cette liste dans notre fonction et on affiche le résultat obtenu.

```
List<int> premierEtDernier(List<int> liste) {
    List<int> resultat = [];
    resultat.add(liste.first);
    resultat.add(liste.last);
    return resultat;
}
```

Run | Debug | Profile

```
✓ void main() {
    List<int> a = [5, 10, 15, 20, 25];
    List<int> nouvelleListe = premierEtDernier(a);

    print("Liste de départ : $a");
    print("Nouvelle liste : $nouvelleListe");
}
```

Résultat :

```
Liste de départ : [5, 10, 15, 20, 25]
Nouvelle liste : [5, 25]
```

Exercice 12 :

Dans cet exercice, on demande à l'utilisateur combien de termes de la suite de Fibonacci il veut afficher, puis on génère ces termes dans une fonction. La suite de Fibonacci commence par 1 et 1, et chaque terme suivant est la somme des deux précédents ($1+1=2$, $1+2=3$, $2+3=5$, etc.). Dans notre fonction `genererFibonacci(int n)`, on traite les cas particuliers : si $n \leq 0$ on renvoie une liste vide, si $n == 1$ on renvoie `[1]`. Sinon, on initialise une liste `suite = [1, 1]`. Ensuite, on fait une boucle `for` qui démarre à l'indice 2 et tourne jusqu'à n . À chaque tour, on calcule la somme des deux derniers éléments (`suite[i - 1] + suite[i - 2]`) et on l'ajoute à la liste avec `suite.add()`. Dans le `main()`, on récupère la saisie de l'utilisateur, on appelle la fonction et on affiche la liste des nombres de Fibonacci.

```
import 'dart:io';

List<int> genererFibonacci(int n) {
  if (n <= 0) {
    return [];
  }
  if (n == 1) {
    return [1];
  }

  List<int> suite = [1, 1];
  for (int i = 2; i < n; i++) {
    int nouveauNombre = suite[i - 1] + suite[i - 2];
    suite.add(nouveauNombre);
  }
  return suite;
}

Run | Debug | Profile
✓ void main() {
  stdout.write("Combien de nombres de Fibonacci voulez-vous générer ? ");
  String saisie = stdin.readLineSync();
  int n = int.parse(saisie);

  List<int> resultat = genererFibonacci(n);
  print("Voici les $n premiers nombres de Fibonacci :");
  print(resultat);
}
```

Résultat :

```
Combien de nombres de Fibonacci voulez-vous générer ? 8
Voici les 8 premiers nombres de Fibonacci :
[1, 1, 2, 3, 5, 8, 13, 21]
```

Exercice 13 :

Ici, on doit créer une fonction qui prend une liste avec des doublons et qui renvoie cette même liste mais débarrassée de tous ses doublons. Pour faire au plus simple sans utiliser de notions compliquées, on crée une fonction `supprimerDoublons(List<int> liste)` dans laquelle on déclare une liste vide `sansDoublons`. On parcourt chaque nombre de la liste de base avec une boucle `for (int element in liste)`. À chaque passage, on vérifie avec `if (!sansDoublons.contains(element))` si le nombre n'est pas déjà présent dans notre nouvelle liste. S'il n'y est pas, on le rajoute avec `add()`, et s'il est déjà présent on ne fait rien. À la fin, on renvoie la liste qui contient tous les éléments uniques. Dans le `main()`, on teste avec une liste contenant plusieurs fois les mêmes chiffres.

```
List<int> supprimerDoublons(List<int> liste) {
    List<int> sansDoublons = [];

    for (int element in liste) {
        if (!sansDoublons.contains(element)) {
            sansDoublons.add(element);
        }
    }

    return sansDoublons;
}

Run | Debug | Profile
✓ void main() {
    List<int> a = [1, 2, 3, 2, 4, 1, 5, 3, 6];
    List<int> resultat = supprimerDoublons(a);

    print("Liste de départ : $a");
    print("Liste sans les doublons : $resultat");
}
```

Résultat :

```
Liste de départ : [1, 2, 3, 2, 4, 1, 5, 3, 6]
Liste sans les doublons : [1, 2, 3, 4, 5, 6]
```

Exercice 14 :

On doit demander une longue chaîne de mots à l'utilisateur et lui réafficher la même chaîne mais avec les mots écrits dans l'ordre inverse. On crée une fonction `inverserPhrase(String phrase)`. Pour séparer les mots, on utilise `phrase.split(' ')` qui découpe la chaîne à chaque espace et nous donne une liste de mots. On crée ensuite une liste vide `motsInverses`. On réutilise la même logique qu'à l'exercice 6 : on fait une boucle `for` qui part du dernier mot (à l'indice `mots.length - 1`) et qui décroît avec `i--` jusqu'à l'indice 0. On ajoute chaque mot dans `motsInverses`. Enfin, on utilise `.join(' ')` pour recoller tous les mots avec un espace entre eux. Dans le `main()`, on demande la phrase et on affiche la phrase inversée.

```
import 'dart:io';

String inverserPhrase(String phrase) {
  List<String> mots = phrase.split(" ");
  List<String> motsInverses = [];

  for (int i = mots.length - 1; i >= 0; i--) {
    motsInverses.add(mots[i]);
  }

  return motsInverses.join(" ");
}

Run | Debug | Profile
✓ void main() {
  stdout.write("Entrez une phrase : ");
  String phrase = stdin.readLineSync()!;

  String resultat = inverserPhrase(phrase);
  print("Phrase inversée :");
  print(resultat);
}
```

Résultat :

```
Entrez une phrase : Mon nom est Michele  
Phrase inversée :  
Michele est mon nom
```

Exercice 15 :

Pour ce générateur de mot de passe, on utilise `dart:io` et `dart:math`. On sépare le code en deux fonctions selon le niveau de sécurité voulu. La première fonction `motDePasseFaible()` pioche deux mots au hasard dans une liste prédéfinie de mots simples (soleil, voiture, pomme, etc.) grâce à `random.nextInt(listeMots.length)` et leur ajoute un nombre aléatoire à la fin. La seconde fonction `motDePasseFort()` utilise une chaîne qui regroupe toutes les lettres minuscules, majuscules, chiffres et symboles. Une boucle `for` tourne 12 fois et pioche à chaque tour un caractère aléatoire qu'on concatène dans une variable `mdp`. Dans le `main()`, on demande à l'utilisateur s'il veut un mot de passe faible ou fort et on lui affiche le mot de passe généré.

```
import 'dart:io';  
import 'dart:math';  
  
String motDePasseFaible() {  
  List<String> listeMots = ["soleil", "pomme", "ordinateur", "voiture", "maison", "plag  
  Random random = Random();  
  String mot1 = listeMots[random.nextInt(listeMots.length)];  
  String mot2 = listeMots[random.nextInt(listeMots.length)];  
  int chiffre = random.nextInt(90) + 10;  
  return "$mot1$mot2$chiffre";  
}  
  
String motDePasseFort() {  
  String caracteres = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@  
  Random random = Random();  
  String mdp = "";  
  for (int i = 0; i < 12; i++) {  
    int index = random.nextInt(caracteres.length);  
    mdp = mdp + caracteres[index];  
  }  
  return mdp;  
}  
  
Run | Debug | Profile  
✓ void main() {  
  print("Générateur de mot de passe :");  
  stdout.write("Voulez-vous un mot de passe faible ou fort ? (tapez 1 pour faible, 2 po  
  String choix = stdin.readLineSync()!;  
  
  if (choix == "1") {  
    String mdp = motDePasseFaible();  
    print("Votre mot de passe faible généré est : $mdp");  
  } else {  
    String mdp = motDePasseFort();  
    print("Votre mot de passe fort généré est : $mdp");  
  }  
}
```

Résultats :

```
Générateur de mot de passe :  
Voulez-vous un mot de passe faible ou fort ? (tapez 1 pour faible, 2 pour fort) : 1  
Votre mot de passe faible généré est : pommevoiture47
```

```
Générateur de mot de passe :  
Voulez-vous un mot de passe faible ou fort ? (tapez 1 pour faible, 2 pour fort) : 2  
Votre mot de passe fort généré est : k9#mP2@vL7!w
```

Exercice 16 :

Dans cet exercice, on crée le jeu des Vaches et des Taureaux. L'ordinateur génère aléatoirement un nombre secret de 4 chiffres entre 1000 et 9999 grâce à `(1000 + random.nextInt(9000)).toString()`. Une vache correspond à un chiffre bien placé et un taureau à un chiffre présent dans le nombre mais au mauvais endroit. On lance une boucle `while (true)` et on utilise un compteur d'essais. À chaque tentative, l'utilisateur saisit un nombre de 4 chiffres. On vérifie d'abord que sa saisie fait bien 4 caractères. Ensuite, on compare chaque chiffre avec une boucle de 0 à 3 : si le chiffre de la saisie est à la même position que celui du secret (`saisie[i] == secret[i]`), on incrémente les vaches, sinon si le secret contient ce chiffre (`secret.contains(saisie[i])`), on incrémente les taureaux. Si l'utilisateur obtient 4 vaches, il a gagné : on affiche un message de victoire avec son nombre d'essais et on stoppe le jeu.

```
Run | Debug | Profile  
✓ void main() {  
    Random random = Random();  
    String secret = (1000 + random.nextInt(9000)).toString();  
    int essais = 0;  
  
    print("Bienvenue dans le jeu des Vaches et Taureaux !");  
    print("Devinez le nombre à 4 chiffres.");  
  
    while (true) {  
        stdout.write("Entrez un nombre à 4 chiffres : ");  
        String saisie = stdin.readLineSync()!;  
  
        if (saisie.length != 4) {  
            print("Erreur : vous devez entrer exactement 4 chiffres !");  
            continue;  
        }  
  
        essais++;  
        int vaches = 0;  
        int taureaux = 0;  
  
        for (int i = 0; i < 4; i++) {  
            if (saisie[i] == secret[i]) {  
                vaches++;  
            } else if (secret.contains(saisie[i])) {  
                taureaux++;  
            }  
        }  
  
        print("$vaches vache(s), $taureaux taureau(x)");  
  
        if (vaches == 4) {  
            print("Bravo ! Vous avez trouvé le nombre $secret en $essais essai(s) !");  
            break;  
        }  
    }  
}
```

Résultat :

```
Bienvenue dans le jeu des Vaches et Taureaux !
Devinez le nombre à 4 chiffres.
Entrez un nombre à 4 chiffres : 1234
1 vache(s), 1 taureau(x)
Entrez un nombre à 4 chiffres : 1563
0 vache(s), 2 taureau(x)
Entrez un nombre à 4 chiffres : 3278
2 vache(s), 0 taureau(x)
Entrez un nombre à 4 chiffres : 3249
4 vache(s), 0 taureau(x)
Bravo ! Vous avez trouvé le nombre 3249 en 4 essai(s) !
```

Exercice 17 :

On doit dessiner un plateau de jeu dans le terminal en fonction de la taille demandée par l'utilisateur. On commence par lui demander la taille qu'il souhaite et on la convertit en entier. En regardant l'exemple, le quadrillage est composé de deux lignes répétées : une ligne horizontale formée de --- et une ligne verticale formée de | terminée par une dernière barre |. En Dart, on peut simplement multiplier une chaîne de caractères par un entier pour la dupliquer (par exemple '---' * taille). Ensuite, on fait une boucle for qui tourne autant de fois que la taille choisie : à chaque itération, elle affiche une ligne horizontale puis une ligne verticale. À la sortie de la boucle, on affiche une dernière ligne horizontale pour fermer le bas de la grille.

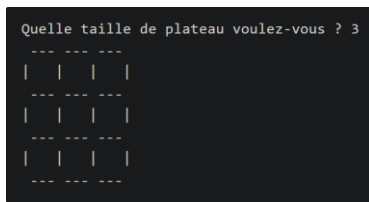
```
import 'dart:io';

Run | Debug | Profile
✓ void main() {
  stdout.write("Quelle taille de plateau voulez-vous ? ");
  String saisie = stdin.readLineSync();
  int taille = int.parse(saisie);

  String ligneHorizontale = " ---" * taille;
  String ligneVerticale = "|   " * taille + "|";

  for (int i = 0; i < taille; i++) {
    print(ligneHorizontale);
    print(ligneVerticale);
  }
  print(ligneHorizontale);
}
```

Résultat :



Exercice 18 :

Ici, le plateau de Tic Tac Toe est représenté sous la forme d'une liste de listes de dimension 3x3, où 0 est une case vide, 1 le joueur 1 et 2 le joueur 2. On écrit une fonction `verifierGagnant` qui vérifie si l'un des joueurs a aligné trois pions identiques. On teste d'abord les trois lignes avec une boucle `for` en vérifiant que la première case n'est pas égale à 0 et qu'elle est identique aux deux autres. On effectue exactement le même test pour les trois colonnes. Ensuite, on vérifie manuellement les deux diagonales ([0][0], [1][1], [2][2] et [0][2], [1][1], [2][0]). Si un alignement est trouvé, la fonction retourne immédiatement le numéro du joueur gagnant. Sinon, elle renvoie 0. Dans le `main()`, on teste avec la matrice fournie dans l'énoncé et on affiche le joueur qui a remporté la partie.

```

    for (int i = 0; i < 3; i++) {
        if (plateau[i][0] != 0 && plateau[i][0] == plateau[i][1] && plateau[i][1] == plateau[i][2])
            return plateau[i][0];
    }
}

// Colonnes
for (int j = 0; j < 3; j++) {
    if (plateau[0][j] != 0 && plateau[0][j] == plateau[1][j] && plateau[1][j] == plateau[2][j])
        return plateau[0][j];
}

// Diagonales
if (plateau[0][0] != 0 && plateau[0][0] == plateau[1][1] && plateau[1][1] == plateau[2][2])
    return plateau[0][0];
}
if (plateau[0][2] != 0 && plateau[0][2] == plateau[1][1] && plateau[1][1] == plateau[2][0])
    return plateau[0][2];
}

return 0;
}

```

Run | Debug | Profile

```

✓ void main() {
    List<List<int>> game = [
        [1, 2, 0],
        [2, 1, 0],
        [2, 1, 1]
    ];

    int gagnant = verifierGagnant(game);

    if (gagnant == 1 || gagnant == 2) {
        print("Le joueur $gagnant a gagné !");
    } else {
        print("Personne n'a gagné pour le moment.");
    }
}

```

Résultat :

Le joueur 1 a gagné !

Exercice 19 :

Pour cet exercice, on s'occupe de la saisie des coordonnées du joueur pour poser un pion sur la grille de Tic Tac Toe. Comme on ne peut pas cliquer avec la souris dans la console, on demande à l'utilisateur le numéro de ligne (0, 1 ou 2) et le numéro de colonne (0, 1 ou 2). On crée un plateau 3x3 initialisé avec des 0. On lit les deux valeurs au clavier avec `stdin.readLineSync()!` et on les convertit avec `int.parse`. On effectue alors deux vérifications : d'abord on s'assure que les coordonnées sont bien comprises entre 0 et 2, puis on contrôle que la case est libre avec `plateau[ligne][colonne] == 0`. Si c'est bon, on place le pion du joueur 1 (valeur 1) et on affiche le plateau mis à jour avec une petite fonction `afficherPlateau`.

```
import 'dart:io';

void afficherPlateau(List<List<int>> plateau) {
  for (int i = 0; i < 3; i++) {
    print(plateau[i]);
  }
}

Run | Debug | Profile
✓ void main() {
  List<List<int>> plateau = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
  ];

  print("Plateau initial :");
  afficherPlateau(plateau);

  stdout.write("Entrez la ligne (0, 1 ou 2) : ");
  int ligne = int.parse(stdin.readLineSync()!);

  stdout.write("Entrez la colonne (0, 1 ou 2) : ");
  int colonne = int.parse(stdin.readLineSync()!);

  if (ligne >= 0 && ligne < 3 && colonne >= 0 && colonne < 3) {
    if (plateau[ligne][colonne] == 0) {
      plateau[ligne][colonne] = 1;
      print("Coup enregistré ! Voici le nouveau plateau :");
      afficherPlateau(plateau);
    } else {
      print("Erreur : cette case est déjà occupée !");
    }
  } else {
    print("Erreur : coordonnées en dehors du plateau !");
  }
}
```

Résultat :

```
Plateau initial :  
[0, 0, 0]  
[0, 0, 0]  
[0, 0, 0]  
Entrez la ligne (0, 1 ou 2) : 1  
Entrez la colonne (0, 1 ou 2) : 1  
Coup enregistré ! Voici le nouveau plateau :  
[0, 0, 0]  
[0, 1, 0]  
[0, 0, 0]
```

Exercice 20 :

Dans cet exercice, on rassemble les trois exercices précédents (le dessin de la grille, la vérification du gagnant et la saisie des coups) pour créer un jeu complet de Tic Tac Toe jouable à 2 joueurs. On crée une fonction `dessinerGrille` qui affiche la grille avec les symboles 'X' pour le joueur 1 et 'O' pour le joueur 2, et une fonction `verifierVictoire`. Dans le `main()`, on utilise une boucle `while` (`coups < 9`) et une variable `joueur` initialisée à 1. À chaque tour, on affiche la grille, on demande au joueur ses coordonnées, on contrôle la validité du coup et on incrémente le nombre de coups. Si la fonction `verifierVictoire` détecte un gagnant, on affiche les félicitations et on arrête la partie avec `return`. Sinon, on passe la main à l'autre joueur avec `joueur = (joueur == 1) ? 2 : 1;`. Si 9 coups sont joués sans vainqueur, la boucle se termine et on affiche qu'il y a match nul.

```

    for (int i = 0; i < 3; i++) {
        print(" --- --- ---");
        String l = "|";
        for (int j = 0; j < 3; j++) {
            String symbole = " ";
            if (p[i][j] == 1) symbole = "X";
            if (p[i][j] == 2) symbole = "O";
            l += " $symbole |";
        }
        print(l);
    }
    print(" --- --- ---");
}

int verifierVictoire(List<List<int>> p) {
    for (int i = 0; i < 3; i++) {
        if (p[i][0] != 0 && p[i][0] == p[i][1] && p[i][1] == p[i][2]) return p[i][0];
        if (p[0][i] != 0 && p[0][i] == p[1][i] && p[1][i] == p[2][i]) return p[0][i];
    }
    if (p[0][0] != 0 && p[0][0] == p[1][1] && p[1][1] == p[2][2]) return p[0][0];
    if (p[0][2] != 0 && p[0][2] == p[1][1] && p[1][1] == p[2][0]) return p[0][2];
    return 0;
}

Run | Debug | Profile
✓ void main() {
    List<List<int>> plateau = [
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]
    ];
    int joueur = 1;
    int coups = 0;

    print("=== JEU DU TIC TAC TOE ===");

    while (coups < 9) {
        dessinerGrille(plateau);
        print("Tour du Joueur $joueur (${joueur == 1 ? 'X' : 'O'})");

        stdout.write("Ligne (0, 1 ou 2) : ");
        String? sL = stdin.readLineSync();
        if (sL == null || sL.isEmpty) break;
        int ligne = int.parse(sL);

        stdout.write("Colonne (0, 1 ou 2) : ");
        String? sC = stdin.readLineSync();
        if (sC == null || sC.isEmpty) break;
        int col = int.parse(sC);

        if (ligne < 0 || ligne > 2 || col < 0 || col > 2 || plateau[ligne][col] != 0) {
            print("Coup invalide ou case déjà occupée ! Réessayez.");
            continue;
        }

        plateau[ligne][col] = joueur;
        coups++;

        int gagnant = verifierVictoire(plateau);
        if (gagnant != 0) {
            dessinerGrille(plateau);
            print("Félicitations ! Le joueur $gagnant a gagné la partie !");
            return;
        }

        joueur = (joueur == 1) ? 2 : 1;
    }

    dessinerGrille(plateau);
    print("Match nul ! Aucun coup restant.");
}

```

Résultat :

```
=== JEU DU TIC TAC TOE ===
--- --- ---
|  |  |  |
--- --- ---
|  |  |  |
--- --- ---
|  |  |  |
--- --- ---
Tour du Joueur 1 (X)
Ligne (0, 1 ou 2) : 0
Colonne (0, 1 ou 2) : 0
--- --- ---
| x |  |  |
--- --- ---
|  |  |  |
--- --- ---
|  |  |  |
--- --- ---
Tour du Joueur 2 (O)
...
--- --- ---
| x | x | x |
--- --- ---
|  | o |  |
--- --- ---
| o |  |  |
--- --- ---
Félicitations ! Le joueur 1 a gagné la partie !
```

Exercice 21 :

Cet exercice fonctionne à l'inverse du jeu du nombre mystère : l'utilisateur choisit un nombre dans sa tête entre 0 et 100, et c'est l'ordinateur qui doit le deviner en posant des questions. Pour trouver la solution le plus rapidement possible, le programme utilise le principe de dichotomie. On définit deux bornes $\text{min} = 0$ et $\text{max} = 100$. À chaque étape, l'ordinateur propose le nombre situé au milieu de l'intervalle avec $(\text{min} + \text{max}) \sim / 2$ (l'opérateur $\sim /$ correspond à la division entière en Dart). L'utilisateur répond alors 'haut' (si la proposition est trop haute), 'bas' (si c'est trop bas) ou 'correct'. Si l'utilisateur répond 'haut', on réduit la borne max ($\text{max} = \text{proposition} - 1$). Si c'est 'bas', on augmente la borne min ($\text{min} = \text{proposition}$).

+1). Dès que l'utilisateur tape 'correct', la boucle s'arrête et le programme affiche le nombre de questions qu'il lui a fallu.

```
import 'dart:io';

Run | Debug | Profile
✓ void main() {
  print("Pensez à un nombre entre 0 et 100 dans votre tête.");
  print("Répondez par : 'haut' (si trop haut), 'bas' (si trop bas) ou 'correct'.");

  int min = 0;
  int max = 100;
  int essais = 0;

  while (true) {
    int proposition = (min + max) ~/ 2;
    essais++;

    stdout.write("Est-ce que c'est $proposition ? ");
    String? reponse = stdin.readLineSync();
    if (reponse == null || reponse.isEmpty) break;
    reponse = reponse.toLowerCase();

    if (reponse == "correct") {
      print("Super ! J'ai trouvé votre nombre $proposition en $essais essai(s) !");
      break;
    } else if (reponse == "haut") {
      max = proposition - 1;
    } else if (reponse == "bas") {
      min = proposition + 1;
    } else {
      print("Réponse non reconnue, merci de taper 'haut', 'bas' ou 'correct'.");
      essais--;
    }
  }
}
```

Résultat :

```
Pensez à un nombre entre 0 et 100 dans votre tête.
Répondez par : 'haut' (si trop haut), 'bas' (si trop bas) ou 'correct'.
Est-ce que c'est 50 ? bas
Est-ce que c'est 75 ? haut
Est-ce que c'est 62 ? bas
Est-ce que c'est 68 ? haut
Est-ce que c'est 65 ? correct
Super ! J'ai trouvé votre nombre 65 en 5 essai(s) !
```

Exercice 22 :

Pour cet exercice, on doit créer une fonction qui compare trois nombres et renvoie le plus grand des trois, mais avec l'interdiction formelle d'utiliser la

méthode max() de Dart. Tout doit être fait à la main avec des variables et des conditions if. On crée donc une fonction trouverPlusGrand(int a, int b, int c). On commence par poser comme hypothèse que le plus grand nombre est le premier en créant une variable int plusGrand = a;. Ensuite, on compare avec le deuxième nombre : si b > plusGrand, alors plusGrand prend la valeur de b. On effectue ensuite la même comparaison avec le troisième nombre c. À la fin, la variable plusGrand contient obligatoirement la valeur la plus élevée, qu'on retourne avec return. Dans le main(), on teste la fonction avec trois valeurs et on affiche le résultat.

```
int trouverPlusGrand(int a, int b, int c) {  
    int plusGrand = a;  
  
    if (b > plusGrand) {  
        plusGrand = b;  
    }  
    if (c > plusGrand) {  
        plusGrand = c;  
    }  
  
    return plusGrand;  
}  
  
Run | Debug | Profile  
✓ void main() {  
    int n1 = 28;  
    int n2 = 75;  
    int n3 = 42;  
  
    int resultat = trouverPlusGrand(n1, n2, n3);  
  
    print("Les trois nombres sont : $n1, $n2 et $n3");  
    print("Le plus grand des trois est : $resultat");  
}
```

Résultat :

```
Les trois nombres sont : 28, 75 et 42  
Le plus grand des trois est : 75
```

Exercice 23 :

Dans cet exercice, on doit piocher un mot au hasard dans un fichier dictionnaire nommé sowpods.txt placé dans le même dossier que notre programme. Pour lire un fichier en Dart, on importe dart:io et on instancie un objet File('sowpods.txt'). Grâce à la fonction readAsLinesSync(), Dart lit automatiquement l'intégralité du fichier et place chaque ligne (donc chaque mot) dans une liste de chaînes List<String> mots. Ensuite, grâce à la bibliothèque dart:math, on tire un indice aléatoire au sort entre 0 et la taille de la liste avec random.nextInt(mots.length). La fonction renvoie le mot situé à cet indice. Dans le main(), on appelle la fonction et on affiche le mot sélectionné dans le terminal.

```
import 'dart:io';
import 'dart:math';

String choisirMotAleatoire(String cheminFichier) {
  File fichier = File(cheminFichier);
  List<String> mots = fichier.readAsLinesSync();
  Random random = Random();
  int index = random.nextInt(mots.length);
  return mots[index];
}

Run | Debug | Profile
✓ void main() {
  String mot = choisirMotAleatoire("sowpods.txt");
  print("Mot sélectionné au hasard dans le dictionnaire SOWPODS :");
  print(mot);
}
```

Résultat :

```
Mot sélectionné au hasard dans le dictionnaire SOWPODS :
EVAPORATE
```

Exercice 24 :

Cet exercice constitue la première partie du jeu du Pendu. Le mot à deviner est fixé à 'EVAPORATE' et le joueur dispose d'essais infinis pour deviner toutes les lettres une par une. On utilise une liste lettresDevinees pour

mémoriser toutes les lettres proposées par le joueur. Dans une boucle infinie `while (true)`, on commence par générer l'affichage du mot masqué : pour chaque lettre du mot secret, si elle est contenue dans `lettresDevinees`, on l'affiche en clair, sinon on affiche un tiret `_`. On vérifie également si le mot est entièrement découvert : si oui, on affiche un message de félicitations et on quitte le jeu. Sinon, on demande une nouvelle lettre au joueur avec `stdin.readLineSync()`! (mise en majuscule avec `.toUpperCase()`). Si le joueur entre une lettre qu'il a déjà testée auparavant, on l'avertit sans le pénaliser. Sinon, on ajoute la lettre à la liste et on lui indique si la lettre est présente ou non.

```
List<String> lettresDevinees = [];

print("Bienvenue dans le jeu du Pendu !");

while (true) {
    String affichage = "";
    bool motComplet = true;

    for (int i = 0; i < motSecret.length; i++) {
        String lettre = motSecret[i];
        if (lettresDevinees.contains(lettre)) {
            affichage += "$lettre ";
        } else {
            affichage += "_ ";
            motComplet = false;
        }
    }

    print("\nMot actuel : $affichage");

    if (motComplet) {
        print("Bravo ! Vous avez deviné le mot entier : $motSecret !");
        break;
    }

    stdout.write("Devinez une lettre : ");
    String? saisie = stdin.readLineSync();
    if (saisie == null || saisie.isEmpty) break;
    String lettreChoisie = saisie[0].toUpperCase();

    if (lettresDevinees.contains(lettreChoisie)) {
        print("Vous avez déjà essayé la lettre '$lettreChoisie' !");
    } else {
        lettresDevinees.add(lettreChoisie);
        if (motSecret.contains(lettreChoisie)) {
            print("Bonne pioche ! La lettre '$lettreChoisie' est bien dans le mot.");
        } else {
            print("Dommage, la lettre '$lettreChoisie' n'est pas dans le mot.");
        }
    }
}
}
```

Résultat :

```
Bienvenue dans le jeu du Pendu !

Mot actuel : _ _ _ _ _
Devinez une lettre : E
Bonne pioche ! La lettre 'E' est bien dans le mot.

Mot actuel : E _ _ _ _ E
Devinez une lettre : A
Bonne pioche ! La lettre 'A' est bien dans le mot.

Mot actuel : E _ A _ _ A _ E
Devinez une lettre : A
Vous avez déjà essayé la lettre 'A' !

...
Mot actuel : E V A P O R A T E
Bravo ! Vous avez deviné le mot entier : EVAPORATE !
```

Exercice 25 :

Ici, on assemble toutes les fonctionnalités pour concevoir le jeu complet du Pendu. On reprend la logique de la partie 1 mais en limitant cette fois le joueur à 6 erreurs maximum. On ajoute également les illustrations du pendu en ASCII art à l'aide de la fonction `afficherPendu(int erreurs)`. Cette fonction stocke une liste de 7 étapes de dessin du gibet et affiche celle qui correspond au nombre d'erreurs commises. Dans le `main()`, on suit le nombre de fautes avec une variable `erreurs`. À chaque tour de boucle, le pendu actuel, les lettres découvertes et le nombre d'erreurs restantes sont affichés. Si le joueur propose une lettre déjà essayée, un message le prévient et on utilise `continue` pour ne pas lui enlever de vie. Si la lettre n'est pas dans le mot, on incrémente `erreurs++`. La partie s'arrête soit lorsque le joueur a découvert toutes les lettres (victoire), soit quand il atteint 6 erreurs (défaite avec révélation du mot secret).

```

import 'dart:io';

void afficherPendu(int erreurs) {
  List<String> etapes = [
    '''
      +---+
      |   |
      |   |
      |   |
      |   |
      |   |
      +---+
    ''',
    '''
      +---+
      |   |
      0   |
      |   |
      |   |
      |   |
      +---+
    ''',
    '''
      +---+
      |   |
      0   |
      |   |
      |   |
      |   |
      +---+
    ''',
    '''
      +---+
      |   |
      0   |
      /|   |
      |   |
      |   |
      +---+
    ''',
  ];
}

```

Résultat :

```
=== JEU DU PENDU COMPLET ===

+---+
|   |
|   |
|   |
|   |
|   |
|   |
+---+

=====

Mot : _ _ _ _ _
Erreurs restantes : 6
Proposez une lettre : E
Bravo, bonne lettre !
...

+---+
|   |
| o  |
|/|\ |
|/\ |
|   |
+---+

=====

Pendu ! Vous avez perdu. Le mot secret était : DEVELOPPEUR
```

Exercice 26 :

Pour ce dernier exercice, on doit stocker les dates d'anniversaire de différentes personnes et permettre à l'utilisateur de les consulter en tapant un nom. Pour représenter ce dictionnaire en Dart, on utilise une `Map<String, String>` où chaque clé est le prénom et le nom d'un personnage célèbre et la valeur associée est sa date de naissance. Au lancement du programme, on affiche un message d'accueil et on liste toutes les personnes enregistrées en parcourant `anniversaires.keys`. On demande ensuite à l'utilisateur d'entrer le nom de la personne souhaitée. Grâce à la méthode `anniversaires.containsKey(saisie)`, on vérifie si la personne existe dans la table. Si elle existe, on récupère sa date d'anniversaire avec `anniversaires[saisie]` et on affiche le texte demandé dans l'énoncé. Si le nom n'est pas trouvé, on affiche un message pour prévenir l'utilisateur.

```
import 'dart:io';

Run | Debug | Profile
✓ void main() {
  Map<String, String> anniversaires = {
    "Albert Einstein": "14/03/1879",
    "Benjamin Franklin": "17/01/1706",
    "Ada Lovelace": "10/12/1815",
  };

  print(">>> Bienvenue dans le dictionnaire d'anniversaire. Nous connaissons les dates
  for (String nom in anniversaires.keys) {
    print(nom);
  }

  stdout.write(">>> De qui voulez vous connaitre la date d'anniversaire ?\n");
  String saisie = stdin.readLineSync()!;

  if (anniversaires.containsKey(saisie)) {
    String date = anniversaires[saisie]!;
    print(">>> $saisie est né le $date.");
  } else {
    print(">>> Désolé, nous n'avons pas la date d'anniversaire de $saisie.");
  }
}
```

Résultat :

```
>>> Bienvenue dans le dictionnaire d'anniversaire. Nous connaissons les dates
d'anniversaire de :
Albert Einstein
Benjamin Franklin
Ada Lovelace
>>> De qui voulez vous connaitre la date d'anniversaire ?
Benjamin Franklin
>>> Benjamin Franklin est né le 17/01/1706.
```

5) Packages Flutter

Les packages Flutter sont des bibliothèques externes disponibles sur le site officiel pub.dev (l'équivalent de npm pour JavaScript). Ils permettent d'ajouter rapidement des fonctionnalités prêtes à l'emploi à notre application. Pour installer un package, on ajoute son nom et sa version dans le fichier `pubspec.yaml` sous la section `dependencies:`, puis on exécute la commande `flutter pub get` pour le télécharger. Enfin, on l'importe en haut de notre fichier Dart avec `import` `'package:nom_du_package/nom_du_package.dart';`.

Exercice 1 : Package english words

On commence par ajouter la dépendance `english_words: ^4.0.0` dans le fichier `pubspec.yaml`, puis on lance `flutter pub get`. On importe ensuite le package dans `main.dart`. Ce package fournit la classe `WordPair` qui permet de générer deux mots anglais aléatoires avec `WordPair.random()`. On stocke cette paire dans une variable `final wordPair` et on affiche le résultat au centre de l'écran grâce à la propriété `.asPascalCase` (qui met la première lettre de chaque mot en majuscule).

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  english_words: ^4.0.0
  google_fonts: ^8.2.1
  animate_do: ^5.1.0

dev_dependencies:
  flutter_test:
    sdk: flutter

flutter:
  uses-material-design: true
```

```
import 'package:flutter/material.dart';
import 'package:english_words/english_words.dart';

Run | Debug | Profile
void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  MyApp({Key? key}) : super(key: key);
  final wordPair = WordPair.random();

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ), // AppBar
        body: Center(
          child: Text(wordPair.asPascalCase),
        ), // Center
      ),
    );
  }
}
```

Résultat :



GrandMess

Exercice 2 : Package Google Fonts

Pour cet exemple, on installe le package `google_fonts: ^2.1.0`. Ce package permet d'utiliser toutes les polices Google Fonts directement dans notre application sans avoir besoin de télécharger les fichiers de police manuellement. Dans le `main.dart`, on importe le package et on applique la police dans la propriété `style` de notre widget `Text()` avec `GoogleFonts.mcLaren()`.

```
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.8  
  english_words: ^4.0.0  
  google_fonts: ^2.1.0
```

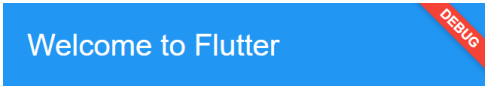
```

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ), // AppBar
        body: Center(
          child: Text(
            'This is Google Fonts',
            style: GoogleFonts.mclaren(),
          ), // Text
        ), // Center
      ), // Scaffold
    ); // MaterialApp
  }
}

```



Welcome to Flutter

This is Google Fonts

Exercice 3 : animate_do

J'ai perdu le code mais j'ai le résultat :

Welcome to Flutter

DEBUG



5) Flutter Dark Mode

Pour gérer le mode sombre avec persistance, on utilise le package `adaptive_theme`. On déclare également les images des icônes de soleil et de lune dans le dossier `assets/icon` :

```
dependencies:
  flutter:
    sdk: flutter
  adaptive_theme: ^3.7.2

flutter:
  uses-material-design: true
  assets:
    - assets/icon/light-icon.png
    - assets/icon/dark-icon.png
```

Ici, on enveloppe notre application avec le widget `AdaptiveTheme`. On lui définit deux thèmes :

```

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return AdaptiveTheme(
      light: ThemeData(
        useMaterial3: false,
        brightness: Brightness.light,
        primarySwatch: Colors.blue,
      ),
      dark: ThemeData(
        useMaterial3: false,
        brightness: Brightness.dark,
        primarySwatch: Colors.orange,
      ),
      initial: AdaptiveThemeMode.light,
      builder: (theme, darkTheme) => MaterialApp(
        debugShowCheckedModeBanner: false,
        title: 'Flutter Dark Mode',
        theme: theme,

```

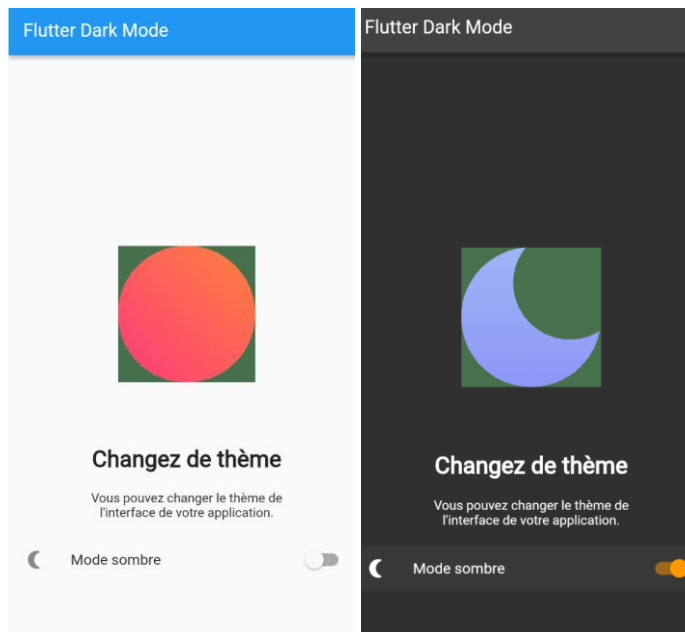
- À l'initialisation (initState), on lit le thème actuellement enregistré via `AdaptiveTheme.getThemeMode()`. Le widget `SwitchListTile` déclenche `AdaptiveTheme.of(context).setDark()` ou `.setLight()` lors du clic. La variable `iconAdress` change pour afficher l'icône correspondante (`lighth-icon.png` ou `dark-icon.png`), et `setState()` rafraîchit l'écran.

```

SwitchListTile(
  title: const Text('Mode sombre'),
  activeColor: Colors.orange,
  secondary: const Icon(Icons.nightlight_round),
  value: darkmode,
  onChanged: (bool value) {
    if (value == true) {
      AdaptiveTheme.of(context).setDark();
      iconAdress = 'assets/icon/dark-icon.png';
    } else {
      AdaptiveTheme.of(context).setLight();
      iconAdress = 'assets/icon/lighth-icon.png';
    }
    setState(() {
      darkmode = value;
    });
  },
), // SwitchListTile

```

Résultats :



6) Conclusion

En conclusion, ce projet m'a permis d'apprendre à maîtriser les fondamentaux du langage Dart à travers de nombreux algorithmes et exercices de programmation, puis de passer concrètement au développement d'applications mobiles avec Flutter. J'ai ainsi appris à structurer une application avec des widgets, à enrichir ses fonctionnalités en intégrant des packages externes comme `google_fonts`, `english_words` ou `animate_do`, et à gérer dynamiquement l'état et l'interface utilisateur avec un `StatefulWidget` pour concevoir une application interactive dotée d'un mode sombre complet et persistant grâce à `adaptive_theme`.